

A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges

Nikhil Anand

1st April, 2026

The rapid rise of AI programming assistants such as GitHub Copilot and ChatGPT has introduced a fundamental shift in how software is developed, transforming programming into a collaborative process between humans and AI systems. These tools are capable of generating code, assisting with debugging, and suggesting implementations, leading to growing interest in their impact on developer productivity and software quality. While some studies highlight their ability to produce high-quality code and improve perceived productivity, others report mixed results regarding actual task completion and code quality. Notably, developers do not accept a large proportion of AI-generated suggestions, indicating a gap between the potential of these tools and their practical usability.

To better understand this gap, Liang et al. conduct a large-scale empirical study investigating how developers use AI programming assistants and the usability challenges they encounter. Through a survey of 410 developers, the study explores usage patterns, motivations, and barriers associated with these tools. The findings reveal that developers primarily use AI assistants for accelerating routine tasks—such as reducing keystrokes and recalling syntax—rather than for higher-level problem solving. At the same time, significant usability issues persist, including difficulties in controlling generated outputs and ensuring that code meets functional and non-functional requirements. These insights highlight the need for improved interaction design and alignment between AI-generated code and developer expectations.

1 Motivation

The rapid adoption of AI programming assistants has significantly reshaped modern software development, prompting researchers to investigate their real-world effectiveness and usability. Tools such as GitHub Copilot and ChatGPT have gained widespread attention due to their ability to generate code and assist developers in various tasks. Prior studies suggest that these systems can produce **high-quality code suggestions** [1, 2], and in some cases are associated with **improvements in developers’ perceived productivity** [3]. However, conflicting evidence exists, as other work reports **no significant improvements in task completion or code quality** when using such tools [4, 5]. This inconsistency motivates a deeper investigation into how these tools perform in practice.

Despite their technical capabilities, developers often do not fully adopt AI-generated suggestions. Empirical findings indicate that only a relatively small proportion of suggestions are accepted in real-world usage—for example, acceptance rates of around **23–29% across common programming languages** [3]. This raises important questions about the usability of AI programming assistants. Prior work suggests that developers may hesitate due to concerns such as **incorrect or low-quality code**, lack of adherence to project-specific requirements, or difficulty in understanding generated outputs [5]. Additionally, developers may struggle with **debugging and adapting AI-generated code**, particularly when it relies on unfamiliar frameworks or APIs [6].

Existing literature has identified several challenges in using AI programming assistants, but it lacks a comprehensive understanding of how frequently these issues occur and which ones matter most in practice. Most prior studies either focus on controlled environments or examine limited aspects of usability. As a result, there is a need to systematically study **the prevalence and importance of usability factors** across a broader developer population. Understanding these aspects is critical because

improvements in model performance alone may not translate into better developer experiences if the tools remain difficult to use or misaligned with developer needs [7].

Motivated by these gaps, the paper aims to provide a large-scale, empirical analysis of how developers interact with AI programming assistants and the challenges they face. By quantifying usability issues and usage patterns, the study seeks to inform both researchers and tool designers about **which aspects of these systems are effective and which require improvement**. Ultimately, this work is driven by the goal of enabling better tool design that reduces cognitive overhead, improves alignment with developer expectations, and increases the practical adoption of AI-assisted programming tools.

1.1 Related Work

The paper situates itself within a growing body of work on the usability of AI programming assistants, spanning both early program synthesis systems and modern transformer-based tools. Prior studies on programming-by-demonstration and neural code generation systems have highlighted usability challenges such as difficulty in correcting generated code and mismatches between user expectations and tool capabilities [8, 9]. Other research has explored the learnability of such tools, particularly for novice programmers, and identified interaction challenges such as refining generated outputs and disambiguating user intent [10, 11]. Compared to these earlier approaches, the current paper focuses specifically on widely deployed, transformer-based assistants like GitHub Copilot and Tabnine, which have demonstrated strong performance in handling both natural language and code inputs [12, 1]. This shift motivates a need to understand usability not just in controlled settings, but in real-world developer workflows.

More recent empirical studies have examined how developers interact with transformer-based AI programming assistants in practice. Research has shown that users often struggle with expressing intent in natural language queries and that generated code may assume familiarity with specific APIs or frameworks [6]. User studies on GitHub Copilot reveal additional challenges, including difficulties in understanding and debugging generated code, as well as varying usage patterns such as “exploration” and “acceleration” modes [5, 13]. Other work has identified common developer activities when using such tools, including verifying suggestions, debugging, and consulting documentation [14]. Large-scale analyses of usage data further indicate relatively low acceptance rates of generated suggestions, despite improvements in perceived productivity [3]. Building on these findings, the current study extends prior work by *quantifying the prevalence and importance of usability challenges across a broader set of tools and a larger developer population*, providing a more comprehensive understanding of how these issues manifest in practice.

2 Methodology

Participants The study recruited a large and diverse group of developers to capture a broad range of experiences with AI programming assistants. Participants were identified from GitHub repositories related to tools such as Copilot and Tabnine, resulting in an initial pool of over **33,000 users**, which was later filtered to **10,530 invited participants** based on available contact information. Ultimately, the survey received **410 responses**, providing a sufficiently large dataset for analysis. The participants represented a wide geographic distribution across **57 countries** and included developers with varying levels of experience, ranging from **1 to 41 years**, with a median of 6 years.

The participant pool also reflected diverse development contexts and technical backgrounds. Respondents reported programming in multiple languages such as **Python, JavaScript, TypeScript, and Java**, and contributed to projects in professional, academic, open-source, and hobby settings. Many participants had direct experience using AI programming assistants, including tools like GitHub Copilot, ChatGPT, and organization-specific systems. This diversity ensures that the findings reflect **real-world usage patterns across different domains and experience levels**, strengthening the external validity of the study.

Survey The authors designed a structured survey to systematically capture developers’ experiences with AI programming assistants. The survey was administered using Qualtrics and took approximately **15 minutes to complete**. It included both **closed-ended questions** (for quantitative analysis) and **open-ended questions** (to capture qualitative insights). Participants were asked to reflect on a specific

project where they used AI tools, helping ground their responses in **concrete, real-world experiences** rather than abstract opinions.

The survey covered multiple aspects of tool usage, including **frequency of use, motivations for using or not using the tools, usability challenges, and reasons for abandoning generated code**. It also included questions about strategies for improving tool output and participant concerns about AI-generated code. To ensure quality, the survey was **piloted with 11 developers** and refined iteratively based on feedback, improving clarity and coverage of usability factors. All questions were optional, and participants were incentivized through a sweepstakes reward.

Analysis The study employed a combination of **quantitative and qualitative analysis techniques** to interpret the survey data. For quantitative analysis, the authors focused on **frequency-based measures**, such as how often participants selected certain responses or rated items as important. These measures were used to estimate the **perceived importance and prevalence** of different usability issues, rather than attempting to capture exact behavioral frequencies.

For qualitative analysis, the authors conducted **open coding on free-text responses**, following established best practices. Multiple rounds of coding were performed, where responses were labeled, merged into themes, and refined into a shared codebook through consensus. This process emphasized **identifying recurring patterns and themes** in developer experiences, such as common frustrations or successful use cases. By combining both analytical approaches, the study provides a **comprehensive and nuanced understanding** of how developers interact with AI programming assistants.

3 Key Findings

3.1 Usage Characteristics

3.1.1 Usage Patterns

The study shows that AI programming assistants are widely used but primarily in a supportive role rather than as primary code generators. Among the tools, GitHub Copilot was the most commonly used, with developers reporting that a median of **30.5% of their code is written with assistance from the tool**. Interestingly, even though tools like ChatGPT had a higher proportion of frequent users, they contributed a smaller portion of actual code generation. Overall, developers tend to use these tools frequently but selectively, leveraging them mainly for **specific parts of development rather than end-to-end coding**. This indicates that AI assistants are integrated into workflows as **partial productivity enhancers rather than complete automation tools**.

3.1.2 Motivation

Developers are primarily motivated to use AI programming assistants for efficiency and convenience. The most important reasons include **reducing keystrokes (autocomplete), finishing tasks faster, and recalling syntax without searching online**. These motivations highlight the role of AI assistants in accelerating routine development tasks. On the other hand, developers are less motivated to use these tools for higher-level activities such as **discovering new solutions or identifying edge cases**. For those who avoid using AI assistants, the main reasons include **generated code not meeting functional or non-functional requirements, difficulty in controlling outputs, and the effort required to debug or modify generated code**. Overall, the findings emphasize that while AI assistants are valued for speed and convenience, their limitations in control and reliability significantly impact adoption.

3.1.3 Successful Use Cases

The study finds that developers are most successful when using AI programming assistants for **repetitive and well-defined tasks**. Common use cases include generating **boilerplate code, CRUD operations, and simple utility functions**, where the logic is straightforward and predictable. These tools are also effective for **autocomplete and small code completions**, allowing developers to quickly

extend patterns they have already started. In such scenarios, AI assistants function as a strong **acceleration mechanism**, helping developers save time without requiring deep reasoning from the model.

Beyond routine tasks, developers also use AI assistants in slightly more exploratory contexts, though with less consistency. These include **generating test cases, handling edge cases, creating proof-of-concepts, and learning new programming languages or APIs**. The tools are particularly useful when developers need a **starting point or reference implementation**, especially in unfamiliar domains. However, their effectiveness decreases as task complexity increases, with developers noting that AI-generated code often struggles with **complex algorithms or non-trivial logic**, reinforcing that current tools are best suited for **simple, structured, and repetitive coding scenarios**.

3.1.4 User Input Strategies

Developers employ several strategies to improve the quality of AI-generated code, with the most common being providing **clear and explicit instructions**. This often involves writing detailed comments, docstrings, or step-by-step descriptions of the intended functionality. Many users also enhance outputs by **adding partial code as context**, allowing the assistant to infer patterns and complete the remaining implementation. Additionally, following **consistent naming conventions and coding styles** helps the model generate more relevant and accurate suggestions.

Another important strategy is breaking down problems into smaller, manageable parts. Developers often **decompose complex tasks into simpler steps** and iteratively refine prompts to guide the assistant more effectively. Some users also rely on **existing project context**, such as open files or previously written code, to improve suggestions. While a subset of developers reported using no specific strategy, others engaged in forms of **prompt engineering**, adjusting inputs dynamically until satisfactory results were obtained. Overall, these strategies highlight that effective use of AI assistants requires **active user involvement and careful input design**, rather than passive reliance on generated outputs.

3.2 Usability of AI Assistants

The study highlights that usability remains a central challenge in the adoption of AI programming assistants, despite their growing popularity. A key issue is the **misalignment between developer intent and generated output**, where suggestions often fail to meet functional or contextual requirements. Developers frequently report that while code may be syntactically correct, it lacks relevance to the specific task or project constraints. This creates additional overhead, as users must refine, correct, or discard suggestions, reducing the overall efficiency gains promised by these tools.

Another major usability concern is the **lack of control and predictability** in how AI assistants generate code. Developers struggle to guide the system toward desired outputs, often needing multiple attempts or rephrasing inputs. The inability to precisely influence results leads to frustration, particularly when dealing with non-trivial tasks. Additionally, users find it difficult to **understand and trust generated code**, especially when it introduces unfamiliar constructs, dependencies, or hidden assumptions. This lack of transparency further complicates debugging and integration into existing codebases.

The study also emphasizes the broader impact of usability issues on developer workflows. AI assistants are most effective when they align with **existing development practices and cognitive processes**, but current limitations often disrupt this alignment. Developers must invest effort in verification, testing, and adaptation, which can offset productivity gains. As a result, usability is not just a secondary concern but a **critical factor determining adoption and long-term effectiveness**. Improving aspects such as contextual awareness, explainability, and interaction design is essential for making these tools more reliable and developer-friendly.

3.3 Additional Feedback

Participants provided additional feedback that sheds light on their expectations and concerns regarding AI programming assistants. A recurring theme is the desire for **greater reliability and accuracy**, with developers expressing frustration over incorrect or incomplete code suggestions. Many users emphasized the importance of tools that can better understand **project-specific context, coding standards,**

and constraints, rather than generating generic solutions. This reflects a need for assistants that can operate more effectively within real-world development environments.

Another key area of feedback relates to the **interaction experience and usability improvements**. Developers expressed interest in features that allow for better control over outputs, such as more interactive refinement mechanisms, explanations of generated code, and the ability to specify constraints more clearly. There is also a demand for improved support in tasks such as **debugging, testing, and maintaining code**, indicating that users want AI assistants to go beyond generation and provide more holistic development support.

Finally, participants raised broader concerns about the long-term implications of using AI programming assistants. These include potential risks such as **over-reliance on AI tools, reduced learning opportunities for developers, and the introduction of subtle bugs or security vulnerabilities**. Some developers also noted ethical and professional considerations, such as code ownership and trust in AI-generated outputs. Overall, the feedback suggests that while developers recognize the benefits of these tools, their future success depends on addressing both **technical limitations and human-centric concerns**.

4 Implications

The paper outlines several directions for future research to build on its findings. One key area is the need for more **longitudinal and observational studies** that capture how developers use AI programming assistants over time. Such studies could provide a deeper understanding of actual usage patterns, complementing self-reported data and addressing limitations related to recall bias. Additionally, future work could explore **controlled experiments** to better measure the impact of these tools on productivity, code quality, and developer behavior.

Another important direction is improving the design and capabilities of AI programming assistants. The study highlights the need for systems that offer **better alignment with developer intent, improved contextual awareness, and greater controllability**. Research could focus on enhancing interaction mechanisms, such as enabling more precise input specifications, providing explanations for generated code, and supporting iterative refinement. These improvements would help address many of the usability challenges identified in the study.

Finally, future work should consider broader implications and emerging use cases of AI-assisted programming. This includes investigating **how these tools affect learning, collaboration, and software engineering practices over time**. There is also a need to study potential risks, such as over-reliance on AI or the introduction of subtle errors, in more depth. By expanding the scope of research, future studies can contribute to developing **more reliable, effective, and human-centered AI programming tools**.

5 Threats to Validity

The study acknowledges several limitations related to its design and data collection. A primary concern is **selection bias**, as participants were recruited from GitHub repositories associated with AI programming assistants, which may overrepresent developers who are already familiar with or favorable toward such tools. Additionally, the response rate is relatively small compared to the number of invited participants, raising the possibility of **non-response bias**. This means that the findings may not fully generalize to the broader population of developers, especially those who do not actively engage with AI tools.

Another threat arises from the reliance on self-reported data. Participants were asked to recall their experiences and estimate usage patterns, which introduces risks of **recall bias and subjective interpretation**. Developers may overestimate or underestimate their reliance on AI assistants or the effectiveness of these tools. Furthermore, since the study captures perceptions rather than direct observations, there is a gap between **reported behavior and actual usage**, which could influence the accuracy of conclusions drawn about productivity and usability.

Finally, there are concerns related to construct and external validity. The survey design, while carefully structured, may not fully capture all relevant aspects of usability, leading to potential **measurement limitations**. Additionally, the study focuses on a specific set of tools and time period, which may limit its applicability as AI programming assistants rapidly evolve. As a result, while the findings provide valuable insights, they should be interpreted with an understanding of these **contextual and methodological constraints**.

References

- [1] Frank F. Xu et al. “A systematic evaluation of large language models of code”. In: *MAPS*. 2022.
- [2] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. “Assessing the quality of Github Copilot’s code generation”. In: *PROMISE*. 2022.
- [3] Albert Ziegler et al. “Productivity assessment of neural code completion”. In: *MAPS*. 2022.
- [4] Saki Imai. “Is GitHub Copilot a substitute for human pair-programming?” In: *ICSE Companion*. 2022.
- [5] Priyan Vaithilingam et al. “Expectation vs. experience: Evaluating usability of code generation tools”. In: *CHI*. 2022.
- [6] Frank F. Xu et al. “In-IDE code generation from natural language: Promise and challenges”. In: *TOSEM* (2022).
- [7] Brad A. Myers et al. “Programmers are users too: Human-centered methods for improving programming tools”. In: *Computer* (2016).
- [8] Kasra Ferdowsifard et al. “Small-step live programming by example”. In: *UIST*. 2020.
- [9] Xi Victoria Lin et al. *Program synthesis from natural language using recurrent neural networks*. Tech. rep. University of Washington, 2017.
- [10] Dhanya Jayagopal et al. “Exploring the learnability of program synthesizers by novice programmers”. In: *UIST*. 2022.
- [11] Andrew McNutt et al. “On the design of AI-powered code assistants for notebooks”. In: *CHI*. 2023.
- [12] Ashish Vaswani et al. “Attention is all you need”. In: *NeurIPS*. 2017.
- [13] Shraddha Barke et al. “Grounded Copilot: How programmers interact with code-generating models”. In: *arXiv* (2022).
- [14] Hussein Mozannar et al. “Reading between the lines: Modeling user behavior and costs in AI-assisted programming”. In: *arXiv* (2022).